

1) Determine if an undirected graph has a cycle

Main idea

Run BFS or DFS on each connected component. In an **undirected** graph, if while exploring u you see a visited neighbor v that is **not** the parent of u , then there is a cycle. This matches the collaboration sheet's BFS/DFS idea.

C++ code (BFS)

```
bool hasCycleUndirected(const Graph& adj) {
    int n = adj.size();
    vector<bool> visited(n, false);
    vector<int> parent(n, -1);
    queue<int> q;

    for (int start = 0; start < n; start++) {
        if (visited[start]) continue;

        visited[start] = true;
        q.push(start);

        while (!q.empty()) {
            int u = q.front();
            q.pop();

            for (int v : adj[u]) {
                if (!visited[v]) {
                    visited[v] = true;
                    parent[v] = u;
                    q.push(v);
                } else if (parent[u] != v) {
                    return true; // visited neighbor that is not parent
                }
            }
        }
    }
    return false;
}
```

Time complexity

Each vertex is visited once and each undirected edge is examined at most twice, so the running time is $O(V + E)$.

2) Given edge $e = (u, v)$, determine if G has a cycle involving e

Main idea

An edge (u, v) lies on a cycle iff after removing that edge, there is still a path from u to v . That is also the idea in the collaboration sheet.

C++ code

```
bool edgeInCycle(const Graph& adj, int a, int b) {
    int n = adj.size();
    vector<bool> visited(n, false);
    queue<int> q;

    visited[a] = true;
    q.push(a);

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        for (int v : adj[u]) {
            // skip the removed edge (a,b) in either direction
            if ((u == a && v == b) || (u == b && v == a)) continue;

            if (!visited[v]) {
                visited[v] = true;
                q.push(v);
            }
        }
    }

    return visited[b];
}
```

Time complexity

One BFS on an adjacency list takes $O(V + E)$.

3) Determine if a directed graph is a DAG

Main idea

A directed graph is a DAG iff it has **no directed cycle**. Use DFS with 3 states:

- 0 = unvisited
- 1 = currently in recursion stack
- 2 = finished

If DFS sees an edge to a vertex in state 1, that is a back edge, so there is a directed cycle.

C++ code

```
bool dfsCycleDirected(int u, const Graph& adj, vector<int>& state) {
    state[u] = 1; // in recursion stack

    for (int v : adj[u]) {
        if (state[v] == 1) return true;          // back edge
        if (state[v] == 0 && dfsCycleDirected(v, adj, state)) {
            return true;
        }
    }

    state[u] = 2; // finished
    return false;
}

bool isDAG(const Graph& adj) {
    int n = adj.size();
    vector<int> state(n, 0);

    for (int i = 0; i < n; i++) {
        if (state[i] == 0) {
            if (dfsCycleDirected(i, adj, state)) {
                return false;
            }
        }
    }
    return true;
}
```

Time complexity

DFS visits each vertex once and each directed edge once, so the running time is **$O(V + E)$** .

5) Determine if vertex v is a cut vertex; if so, find a disconnected pair

This is Question 5. The collaboration doc gives the simple brute-force BFS/DFS removal idea, which is very exam-friendly.

Main idea

Temporarily remove vertex `cut`. Run BFS from any other vertex. If some remaining vertex is unreachable, then `cut` is a cut vertex. The disconnected pair is:

- u = the BFS start vertex
- w = any unvisited vertex

C++ code

```
pair<bool, pair<int,int>> isCutVertex(const Graph& adj, int cut) {
    int n = adj.size();
    vector<bool> visited(n, false);
    queue<int> q;

    int start = -1;
    for (int i = 0; i < n; i++) {
        if (i != cut) {
            start = i;
            break;
        }
    }

    if (start == -1) return {false, {-1, -1}}; // graph too small

    visited[start] = true;
    q.push(start);

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        for (int v : adj[u]) {
            if (v == cut) continue; // skip removed vertex
            if (!visited[v]) {
                visited[v] = true;
                q.push(v);
            }
        }
    }

    for (int i = 0; i < n; i++) {
        if (i != cut && !visited[i]) {
            return {true, {start, i}};
        }
    }

    return {false, {-1, -1}};
}
```

Time complexity

One BFS while skipping one vertex still takes $O(V + E)$.

6) Why Dijkstra fails with negative edges

Main idea

Dijkstra's proof assumes that when a vertex is chosen as the current minimum-distance unprocessed vertex, its shortest-path value is already final. With a negative edge, a later path can reduce that value, so the greedy choice is no longer safe. That is exactly the reason summarized in the collaboration sheet.

Tiny example

```
s -> a (2)
s -> b (5)
b -> a (-10)
```

Dijkstra may finalize a with distance 2, but the true shortest path is:

```
s -> b -> a = 5 + (-10) = -5
```

So the earlier “finalized” answer was wrong.

Time complexity

This is a proof/explanation question, not really a coding question. If they ask about standard Dijkstra runtime, use whatever version your class uses. The collaboration notes mention the array-based version.

8) Construct the reverse of a directed graph

The collaboration sheet lists this as remaining Question 8, though the bank text also shows the reverse-graph problem under another numbering spot; the actual content is the same reverse-edges task.

Main idea

Create a new adjacency list `rev`. For every directed edge `u -> v` in `adj`, add `u` to `rev[v]`.

C++ code

```
Graph reverseGraph(const Graph& adj) {
    int n = adj.size();
    Graph rev(n);

    for (int u = 0; u < n; u++) {
        for (int v : adj[u]) {
            rev[v].push_back(u);
        }
    }

    return rev;
}
```

Time complexity

We scan each vertex and each outgoing edge once, so the running time is $O(V + E)$. The collaboration sheet states the same edge-reversal idea and gives linear-in-edges work.

11) Find a pair (u, v) with the largest number of common neighbors

Main idea

Try every pair of vertices (u, v). For each pair, count how many common neighbors they have. Keep the pair with the largest count. The collaboration notes settle on $O(V^3)$ worst-case for the simple approach.

C++ code

```
pair<int,int> maxCommonNeighbors(const Graph& adj) {
    int n = adj.size();
    pair<int,int> best = {-1, -1};
    int bestCount = -1;

    vector<bool> mark(n, false);
    vector<int> used; // to unmark only what we marked

    for (int u = 0; u < n; u++) {
        for (int v = u + 1; v < n; v++) {
            used.clear();

            for (int x : adj[u]) {
                if (!mark[x]) {
                    mark[x] = true;
                    used.push_back(x);
                }
            }

            int count = 0;
            for (int y : adj[v]) {
                if (mark[y]) count++;
            }

            if (count > bestCount) {
                bestCount = count;
                best = {u, v};
            }

            for (int x : used) mark[x] = false;
        }
    }

    return best;
}
```

Time complexity

There are $O(V^2)$ vertex pairs. For each pair, we may scan up to $O(V)$ neighbors in the worst case, so total is $O(V^3)$.

14) Determine all apex species

Main idea

A directed edge (u, v) means u eats v . So an apex species is a vertex with **indegree 0**, meaning no one eats it. The collaboration sheet suggests marking every second endpoint as non-apex.

C++ code

```
vector<int> findApexSpecies(const Graph& adj) {
    int n = adj.size();
    vector<int> indegree(n, 0);

    for (int u = 0; u < n; u++) {
        for (int v : adj[u]) {
            indegree[v]++;
        }
    }

    vector<int> apex;
    for (int i = 0; i < n; i++) {
        if (indegree[i] == 0) {
            apex.push_back(i);
        }
    }

    return apex;
}
```

Time complexity

Computing indegrees takes $O(V + E)$, and scanning for zeros takes $O(V)$, so total is $O(V + E)$.

17) Determine all sink vertices that lose power when station s fails

Main idea

A sink t loses power after removing s iff in the graph with s removed, **no source can reach t** . So:

1. compute all source vertices and sink vertices,
2. run one multi-source BFS/DFS from all sources except s ,
3. never traverse through s ,
4. any sink not reached is a failed sink.

The collaboration sheet proposes a DFS-marking approach with the same intended $O(V+E)$ goal.

C++ code

```
vector<int> sinksLosingPower(const Graph& adj, int s) {
    int n = adj.size();
    vector<int> indegree(n, 0), outdegree(n, 0);

    for (int u = 0; u < n; u++) {
        outdegree[u] = (int)adj[u].size();
        for (int v : adj[u]) indegree[v]++;
    }
    vector<bool> visited(n, false);
    queue<int> q;

    for (int i = 0; i < n; i++) { // Start BFS from every source except s
        if (i != s && indegree[i] == 0) {
            visited[i] = true;
            q.push(i);
        }
    }

    while (!q.empty()) {
        int u = q.front(); q.pop();

        for (int v : adj[u]) {
            if (v == s) continue; // s has failed, cannot transmit through it
            if (!visited[v]) {
                visited[v] = true;
                q.push(v);
            }
        }
    }

    vector<int> failedSinks;
    for (int i = 0; i < n; i++) {
        if (i != s && outdegree[i] == 0 && !visited[i]) {
            failedSinks.push_back(i);
        }
    }
    return failedSinks;
}
```

Time complexity

Computing indegree/outdegree is $O(V + E)$, the multi-source BFS is $O(V + E)$, and the final sink scan is $O(V)$. Total: $O(V + E)$.

18) Determine if a directed graph is semi connected

Main idea

The simplest correct exam answer is:

- for each vertex u , run BFS/DFS,
- record which vertices are reachable from u ,
- for every pair (u, v) , check that either u reaches v or v reaches u .

That is easy to explain and always correct.

C++ code

```
bool isSemiconnected(const Graph& adj) {
    int n = adj.size();
    vector<vector<bool>> reach(n, vector<bool>(n, false));

    for (int s = 0; s < n; s++) {
        queue<int> q;
        reach[s][s] = true;
        q.push(s);

        while (!q.empty()) {
            int u = q.front();
            q.pop();

            for (int v : adj[u]) {
                if (!reach[s][v]) {
                    reach[s][v] = true;
                    q.push(v);
                }
            }
        }
    }

    for (int u = 0; u < n; u++) {
        for (int v = u + 1; v < n; v++) {
            if (!reach[u][v] && !reach[v][u]) {
                return false;
            }
        }
    }

    return true;
}
```

Time complexity

We run BFS from every vertex. Each BFS takes $O(V + E)$, so total is $O(V(V + E))$.

19) Compute the diameter of an undirected graph, and a faster version for a tree

- 19a) Diameter of a general undirected graph

Main idea

Run BFS from every vertex. The farthest distance found from any BFS is the diameter.

C++ code

```
int bfsFarthestDistance(const Graph& adj, int start) {
    int n = adj.size();
    vector<int> dist(n, -1);
    queue<int> q;

    dist[start] = 0;
    q.push(start);

    int best = 0;

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        for (int v : adj[u]) {
            if (dist[v] == -1) {
                dist[v] = dist[u] + 1;
                best = max(best, dist[v]);
                q.push(v);
            }
        }
    }

    return best;
}

int graphDiameter(const Graph& adj) {
    int n = adj.size();
    int diameter = 0;

    for (int s = 0; s < n; s++) {
        diameter = max(diameter, bfsFarthestDistance(adj, s));
    }

    return diameter;
}
```

Time complexity

We run BFS v times, each in $O(V + E)$, so total is $O(V(V + E))$.

- 19b) Diameter of a tree

Main idea

Use the classic two-BFS trick:

1. BFS from any vertex r and find a farthest vertex a
2. BFS from a and find a farthest vertex b
3. $\text{dist}(a, b)$ is the tree diameter

C++ code

```
pair<int,int> bfsFarthestVertex(const Graph& adj, int start) {
    int n = adj.size();
    vector<int> dist(n, -1);
    queue<int> q;

    dist[start] = 0;
    q.push(start);

    int farthest = start;

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        for (int v : adj[u]) {
            if (dist[v] == -1) {
                dist[v] = dist[u] + 1;
                q.push(v);
                if (dist[v] > dist[farthest]) {
                    farthest = v;
                }
            }
        }
    }

    return {farthest, dist[farthest]};
}

int treeDiameter(const Graph& tree) {
    auto first = bfsFarthestVertex(tree, 0);
    auto second = bfsFarthestVertex(tree, first.first);
    return second.second;
}
```

Time complexity

A tree has $E = V - 1$, and we do only two BFS traversals, so the running time is **$O(V)$** .
